

¿Qué es una Base de Datos?

Una base de datos es una recopilación organizada de información o datos estructurados, que normalmente se almacena de forma electrónica en un sistema informático. Una base de datos está controlada por un Sistema de Gestión de Bases de Datos (DBMS). En conjunto, los datos y el DBMS, junto con las aplicaciones que están asociados a ellos, reciben el nombre de sistema de bases de datos.

Bases de Datos Relacionales y SQL

El tipo más común de base de datos en el entorno corporativo es la **Base de Datos Relacional (RDBMS)**. Este tipo organiza los datos en tablas con columnas y filas (muy similar a una hoja de cálculo). Las diferentes tablas pueden "relacionarse" entre sí a partir de datos comunes.

Para interactuar con estas bases de datos utilizamos **SQL (Structured Query Language)**, el lenguaje estándar para almacenar, manipular y recuperar datos.

¿Qué es MySQL?

MySQL es uno de los sistemas de gestión de bases de datos relacionales de código abierto más populares del mundo. Desarrollado por Oracle, es reconocido por su velocidad, confiabilidad y facilidad de uso. Se utiliza habitualmente en aplicaciones web, actuando como el componente de base de datos en el famoso stack LAMP (Linux, Apache, MySQL, PHP/Python/Perl).

Glosario Fundamental

- **Tabla:**
Estructura principal donde se almacena la información. Ej: una tabla llamada "Usuarios".
- **Fila (Registro):**
Una entrada individual en una tabla. Ej: Los datos del usuario "Juan Pérez".
- **Columna (Campo):**
Un atributo específico de los datos en la tabla. Ej: "Nombre", "Email", "Edad".

Llaves y Relaciones

Para que las tablas puedan conectarse entre sí de manera eficiente y asegurar que no haya datos duplicados o registros huérfanos, MySQL utiliza el concepto de "Llaves" (Keys).

- **Llave Primaria (Primary Key - PK):** Es una columna (o conjunto de columnas) que identifica de manera *única* cada fila en una tabla. No puede haber dos filas con la misma llave primaria, y no puede ser nula (vacía). Ejemplo: `id_usuario`.
- **Llave Foránea (Foreign Key - FK):** Es una columna en una tabla que hace referencia a la Llave Primaria de *otra* tabla. Esto es lo que crea la "relación" entre ambas.

Tipos de Datos en MySQL

Al crear una tabla, debemos definir qué tipo de información almacenará cada columna. Esto optimiza el espacio y previene errores (por ejemplo, intentar guardar un texto en un campo de edad).

Tipo	Clasificación	Descripción y Uso Común
INT	Numérico (Entero)	Números enteros sin decimales. Rango de -2147483648 a 2147483647. Ideal para edades, cantidades y Llaves Primarias (IDs).
DECIMAL(M,D)	Numérico (Exacto)	Números con decimales fijos (ej. moneda). M = total de dígitos, D = decimales. Ej: DECIMAL(10,2) para precios.
VARCHAR(N)	Cadena de texto	Texto de longitud variable hasta N caracteres (máx. 65535). Ideal para nombres, correos electrónicos y direcciones.
TEXT	Cadena de texto	Para textos largos que exceden los límites comunes del VARCHAR. Útil para artículos, descripciones de productos o comentarios.
DATE	Fecha	Almacena una fecha en formato 'YYYY-MM-DD'. Ej: Fechas de nacimiento.
DATETIME	Fecha y Hora	Almacena fecha y hora en formato 'YYYY-MM-DD HH:MM:SS'. Útil para registrar cuándo se realizó una compra.
BOOLEAN	Lógico	En MySQL se traduce como TINYINT(1). Almacena 1 (Verdadero/True) o 0 (Falso/False). Útil para estados ("activo", "pagado").

Introducción a SQL: Operaciones CRUD

En el mundo de las bases de datos, las siglas **CRUD** representan las cuatro operaciones básicas que podemos realizar sobre los datos: **Create** (Crear), **Read** (Leer), **Update** (Actualizar) y **Delete** (Eliminar).

1. Crear una Tabla (CREATE TABLE)

Antes de insertar datos, necesitamos estructurar nuestra tabla definiendo sus columnas y tipos de datos.

```
-- Crear una tabla para almacenar información de clientes CREATE TABLE clientes (  
id INT AUTO_INCREMENT PRIMARY KEY, nombre VARCHAR(100) NOT NULL, email VARCHAR(100)  
UNIQUE, fecha_registro DATE );
```

2. Insertar Datos (INSERT INTO - Create)

Una vez creada la tabla, utilizamos la sentencia `INSERT INTO` para agregar nuevas filas (registros).

```
-- Insertar un solo registro. El 'id' se genera automáticamente. INSERT INTO  
clientes (nombre, email, fecha_registro) VALUES ('Ana López', 'ana@ejemplo.com',  
'2026-05-10'); -- Insertar múltiples registros a la vez INSERT INTO clientes  
(nombre, email, fecha_registro) VALUES ('Carlos Ruiz', 'carlos@ejemplo.com', '2026-  
05-11'), ('María Gómez', 'maria@ejemplo.com', '2026-05-12');
```

3. Consultar Datos (SELECT - Read)

La operación más común es leer datos. `SELECT` permite recuperar columnas específicas o todas las columnas (usando `*`) de una tabla.

```
-- Seleccionar todas las columnas y todos los registros SELECT * FROM clientes; --  
Seleccionar columnas específicas SELECT nombre, email FROM clientes; -- Filtrar  
resultados con la cláusula WHERE SELECT * FROM clientes WHERE id = 1;
```

4. Actualizar Datos (UPDATE)

Se utiliza para modificar los valores de registros que ya existen en la tabla. Es vital utilizar la cláusula WHERE para no actualizar toda la tabla por error.

```
-- Cambiar el correo electrónico del cliente con id 2 UPDATE clientes SET email =  
'nuevo_correo_carlos@ejemplo.com' WHERE id = 2;
```

5. Eliminar Datos (DELETE)

Borra filas de una tabla de forma permanente. Al igual que con UPDATE, el WHERE es fundamental.

```
-- Eliminar al cliente que tiene el id 3 DELETE FROM clientes WHERE id = 3;
```

¡Precaución con UPDATE y DELETE!

Si ejecutas un comando UPDATE o DELETE sin escribir la cláusula WHERE, la acción afectará a **TODAS**

las filas de tu tabla. Ej: DELETE FROM clientes; vaciará toda la tabla.

Cláusulas Avanzadas de Consulta

Además de recuperar datos básicos, SQL nos permite ordenar y agrupar la información resultante para realizar análisis.

- **ORDER BY:** Ordena los resultados por una o más columnas, ya sea de forma ascendente (ASC, por defecto) o descendente (DESC).
- **LIMIT:** Restringe el número de filas que devolverá la consulta. Muy útil para la paginación.

```
-- Obtener los 10 clientes más recientes ordenados por fecha de registro SELECT  
nombre, fecha_registro FROM clientes ORDER BY fecha_registro DESC LIMIT 10;
```

Trabajando con Múltiples Tablas: Los JOINS

La verdadera potencia de una base de datos *relacional* radica en su capacidad de conectar datos dispersos en diferentes tablas. La cláusula JOIN se utiliza para combinar filas de dos o más tablas, basándose en una columna común entre ellas (normalmente relacionando una Llave Foránea con una Llave Primaria).

Tipos principales de JOIN

- **INNER JOIN:**

Devuelve únicamente las filas que tienen una coincidencia en **ambas** tablas. Es el más común.

- **LEFT JOIN:**

Devuelve todas las filas de la tabla de la izquierda (la primera que se menciona), y las filas coincidentes de la tabla de la derecha. Si no hay coincidencia, devuelve valores NULL para el lado derecho.

- **RIGHT JOIN:**

Igual que el LEFT, pero devuelve todo de la tabla de la derecha.

Ejemplo Práctico de INNER JOIN

Imagina que tenemos la tabla `clientes` (creada anteriormente) y una nueva tabla `pedidos`. Queremos saber qué compró cada cliente consultando ambas tablas al mismo tiempo.

```
-- Estructura de la tabla pedidos (referenciando al cliente) CREATE TABLE pedidos (
id_pedido INT AUTO_INCREMENT PRIMARY KEY, cliente_id INT, producto VARCHAR(100),
monto DECIMAL(10,2), FOREIGN KEY (cliente_id) REFERENCES clientes(id) );
```

Ahora, utilizamos `INNER JOIN` para juntar la información del cliente con el detalle de su pedido:

```
-- Obtener el Nombre del cliente, el Producto que compró y el Monto SELECT
clientes.nombre, pedidos.producto, pedidos.monto FROM clientes INNER JOIN pedidos
ON clientes.id = pedidos.cliente_id;
```

Explicación: Le decimos a MySQL: "Tráeme columnas de `clientes` y de `pedidos`. Conecta ambas tablas basándote en la regla `ON`: el ID principal del cliente debe ser exactamente igual al ID guardado en el registro del pedido". El resultado será una única tabla consolidada y lista para analizar.